



Pandas – Step by Step Journey

Pandas





Introduction to Pandas

- **Pandas overview:** A powerful Python library for data manipulation and analysis.
- **Real-world example:** Analyzing sales data from an e-commerce website to identify trends and customer behavior.
- **Importance in AI:** Pandas facilitates data preprocessing tasks essential for building machine learning models.





Series and DataFrames

- Explanation of Series and DataFrames as fundamental data structures in Pandas.
- Real-world example: Using a DataFrame to organize and analyze sensor data collected from IoT devices in smart homes.
- Understanding Series and DataFrames is crucial for handling and processing datasets in AI projects.



Series and DataFrames










```

main.py
1  import pandas as pd
2
3  # create a dictionary
4  data = {'Name': ['John', 'Alice', 'Bob'],
5          'Age': [25, 30, 35],
6          'City': ['New York', 'London', 'Paris']}
7
8  # create a dataframe from the dictionary
9  df = pd.DataFrame(data)
10
11  print(df)
12

```




Save

Run

Shell

	Name	Age	City
0	John	25	New York
1	Alice	30	London
2	Bob	35	Paris

> |



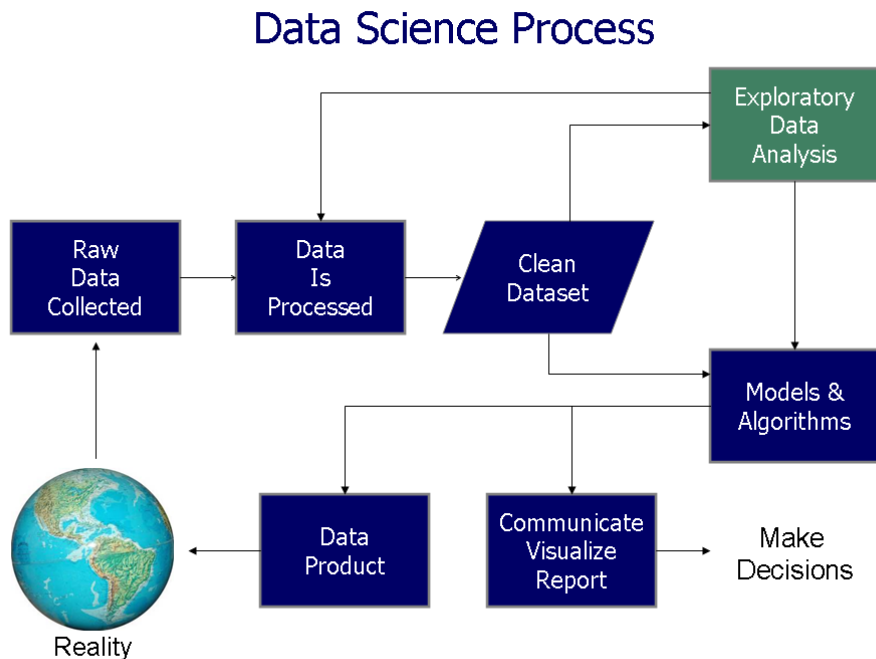


Data Exploration and Inspection

- Methods for exploring and inspecting data: `head()`, `tail()`, `info()`, `describe()`.
- Real-world example: Using `describe()` to generate summary statistics for a dataset containing housing prices.
- Data exploration helps AI students understand the characteristics and distributions of data.



Data Exploration and Inspection





Indexing and Selection

- Accessing elements using `.loc[]` and `.iloc[]` for row and column selection.
- Real-world example: Extracting specific rows and columns from a DataFrame containing weather data for analysis.
- Understanding indexing and selection is crucial for data manipulation and feature selection in AI projects.





Indexing and Selection



```
main.py  [ ] [ ] Save Run

1  # importing pandas package
2  import pandas as pd
3
4  # making data frame from csv file
5  data = pd.read_csv("nba.csv", index_col = "Name")
6
7  # retrieving columns by indexing operator
8  first = data["Age"]
9
10
11
12  print(first)
13
```



Indexing and Selection

```

Name
Avery Bradley      25.0
Jae Crowder        25.0
John Holland       27.0
R.J. Hunter        22.0
Jonas Jerebko      29.0
Amir Johnson       29.0
Jordan Mickey      21.0
Kelly Olynyk       25.0
Terry Rozier       22.0
Marcus Smart       22.0
Jared Sullinger    24.0
Isaiah Thomas      27.0
▪                  ▪
▪                  ▪
▪                  ▪
Joe Ingles         28.0
Chris Johnson      26.0
Trey Lyles         20.0
Shelvin Mack       26.0
Raul Neto          24.0
Tibor Pleiss       26.0
Jeff Withey        26.0
NaN               NaN
Name: Age, Length: 458, dtype: float64
  
```





Data Cleaning and Handling Missing Values

- Techniques for handling missing data: `dropna()`, `fillna()`.
- Real-world example: Cleaning a dataset of patient records by removing or imputing missing values before analysis.
- Data cleaning ensures the quality and integrity of data used in AI algorithms.



Data Cleaning and Handling Missing Values









main.py


Save
Run

```

1 import pandas as pd
2 import numpy as np
3
4 # create dataframe with missing values
5 data = {
6     'A': [1, 2, np.nan, 4, 5],
7     'B': [np.nan, 2, 3, 4, 5],
8     'C': [1, 2, 3, np.nan, 5],
9     'D': [1, 2, 3, 4, 5]
10 }
11 df = pd.DataFrame(data)
12
13 print(df)
  
```

Shell

	A	B	C	D
0	1.0	NaN	1.0	1
1	2.0	2.0	2.0	2
2	NaN	3.0	3.0	3
3	4.0	4.0	NaN	4
4	5.0	5.0	5.0	5

>





Data Manipulation and Transformation

- Applying functions to data: `apply()`, `map()`, `applymap()`.
- Real-world example: Converting temperature values from Fahrenheit to Celsius using `apply()` on a DataFrame column.
- Data manipulation and transformation are essential for feature engineering in AI model building.



Data Manipulation and Transformation











main.py




Save

Run

```

1 import pandas as pd
2
3 # define a dictionary containing student data
4 data = {'Name': ['John', 'Emma', 'Michael', 'Sophia'],
5         'Height': [5.5, 6.0, 5.8, 5.3],
6         'Qualification': ['BSc', 'BBA', 'MBA', 'BSc']}
7
8 # convert the dictionary into a DataFrame
9 df = pd.DataFrame(data)
10
11 # declare a new list
12 address = ['New York', 'London', 'Sydney', 'Toronto']
13
14 # assign the list as a column
15 df['Address'] = address
16
17 print(df)

```

Shell

	Name	Height	Qualification	Address
0	John	5.5	BSc	New York
1	Emma	6.0	BBA	London
2	Michael	5.8	MBA	Sydney
3	Sophia	5.3	BSc	Toronto

> |





Grouping and Aggregation

- Grouping data using `groupby()` for statistical analysis and aggregation.
- Real-world example: Grouping customer purchase data by product category to calculate total sales revenue.
- Grouping and aggregation help derive meaningful insights from data in AI-driven business decisions.



Grouping and Aggregation










main.py




Save

Run

```

1  # import module
2  import pandas as pd
3
4  # Creating our dataset
5  df = pd.DataFrame([[9, 4, 8, 9],
6                    [8, 10, 7, 6],
7                    [7, 6, 8, 5]],
8                    columns=['Maths', 'English',
9                             'Science', 'History'])
10
11 # display dataset
12 print(df)
13

```

Shell

```

Maths  English  Science  History
0      9      4      8      9
1      8     10     7      6
2      7      6      8      5
> |

```





Visualization with Pandas

- Plotting data directly from DataFrames using `plot()` for visual analysis.
- Real-world example: Visualizing trends in temperature data over time using line plots generated from a DataFrame.
- Data visualization aids in understanding patterns and relationships in data for AI model interpretation.





Visualization with Pandas



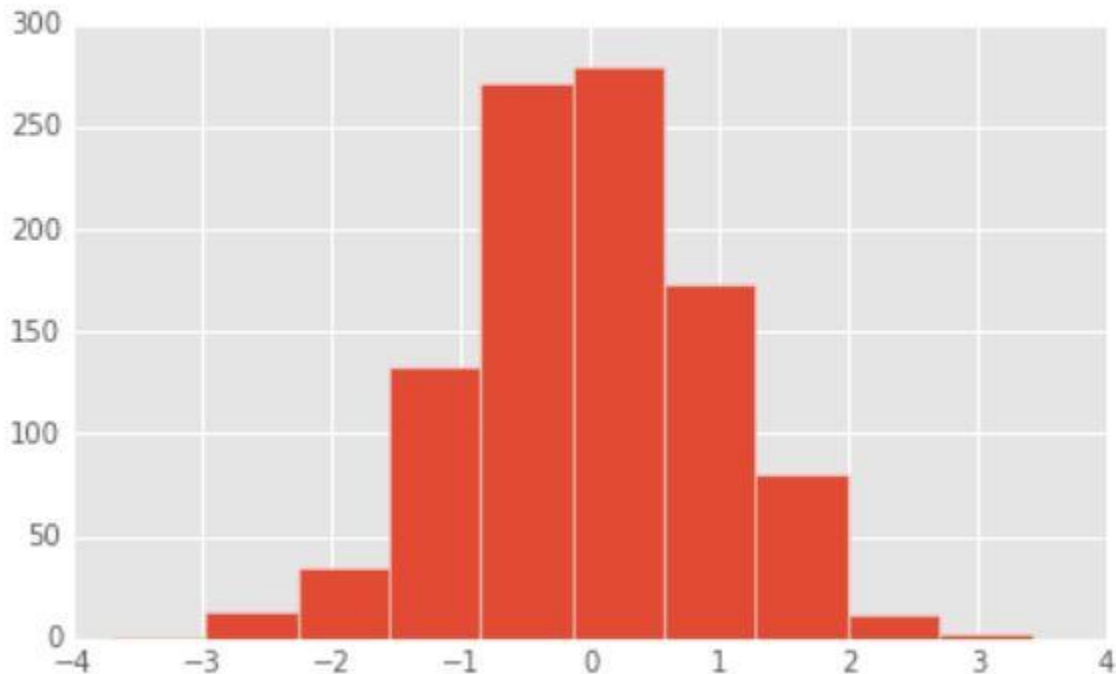
```
main.py   Save Run

1 import numpy as np
2 import pandas as pd
3
4 # There are some fake data csv files
5 # you can read in as dataframes
6 df1 = pd.read_csv('df1', index_col=0)
7 df2 = pd.read_csv('df2')
8
```



Visualization with Pandas

<matplotlib.axes._subplots.AxesSubplot at 0x125853940>





Reshaping and Pivoting Data

- Reshaping data using `pivot()` and `melt()` for restructuring datasets.
- Real-world example: Pivoting customer transaction data to analyze purchase patterns and customer behavior.
- Reshaping data facilitates data aggregation and analysis in AI-driven business insights.



Reshaping and Pivoting Data










main.py




Save

Run

```

1 import pandas as pd
2
3 # create a DataFrame
4 data = {'Date': ['2023-08-01', '2023-08-01', '2023-08-02', '2023-08-02'],
5         'Category': ['A', 'B', 'A', 'B'],
6         'Value': [10, 20, 30, 40]}
7 df = pd.DataFrame(data)
8
9 print("Original Dataframe:\n", df)
10
11 # pivot the DataFrame
12 pivot_df = df.pivot(index='Date', columns='Category', values='Value')
13 print("Reshaped DataFrame:\n", pivot_df)

```

Shell

```

Original Dataframe:
Date Category  Value
0 2023-08-01      A    10
1 2023-08-01      B    20
2 2023-08-02      A    30
3 2023-08-02      B    40

Reshaped DataFrame:
Category      A      B
Date
2023-08-01    10     20
2023-08-02    30     40
> |

```





Handling Excel Files with Pandas

- Reading and writing Excel files with Pandas: `read_excel()`, `to_excel()`.
- Real-world example: Importing financial data from Excel spreadsheets for analysis and forecasting.
- Handling Excel files enables seamless integration of external data sources in AI projects.





Case Study: Analyzing Customer Data

- Applying Pandas techniques to analyze customer behavior and preferences.
- Real-world example: Analyzing e-commerce sales data to identify customer segments and personalize marketing strategies.
- Case studies provide practical insights into applying Pandas skills in real-life AI projects.





Case Study: Social Media Analytics

- Leveraging Pandas for analyzing social media engagement metrics and trends.
- Real-world example: Analyzing Twitter data to measure brand sentiment and track campaign performance.
- Social media analytics with Pandas helps businesses optimize marketing strategies and brand perception in AI-driven digital marketing.





Good Luck!!

Thanks

